Programming Languages

Structure of a C Program

A C program is generally divided into the following sections:

1. Documentation Section

Contains comments that describe the program.

/* Program to add two numbers */

2. Link Section

Contains the header files required by the program.

#include <stdio.h>

#include <math.h>

3. Definition Section

Contains symbolic constants defined using #define.

#define PI 3.14

4. Global Declaration Section

Contains global variables and function declarations that can be accessed throughout the program.

int x;

void display();

5. main() Function

The main() function is the **entry point of a C program**.

It generally contains:

- Declaration of local variables
- Executable statements
- return statement

```
int main()
{
   int a, b, sum;
   a = 10;
   b = 20;
   sum = a + b;
   printf("%d", sum);
   return 0;
}
```

6. User-Defined Function Section

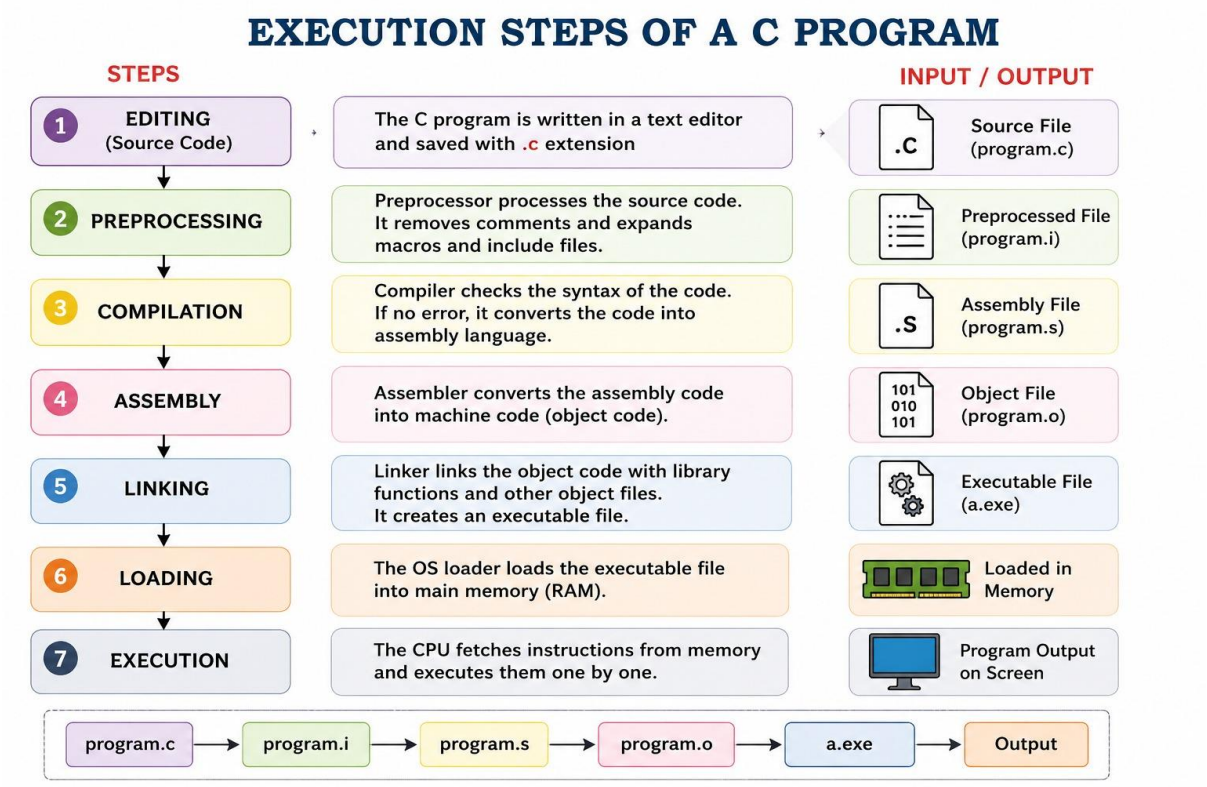
Contains functions defined by the programmer.

```
void display()
{
   printf("Hello");
}
```

General Structure

```
/* Documentation Section */
#include <stdio.h>              // Link Section
#define PI 3.14                 // Definition Section
int x;                          // Global Declaration
void display();                 // Function Declaration
int main()                      // Main Function
{
   int a, b;                    // Local Declaration
```

```
    a = 10;                         // Executable Statements
    b = 20;
    printf("%d", a + b);
    return 0;
}
void display()                      // User-Defined Function
{
    printf("Hello");
}
```



Compilation and Execution in C

1. Compilation

Compilation is the process of translating a C source program into machine-understandable code through several stages.

The C source program is written in a file with .c extension.

Source Program (.c)

↓

Preprocessor

↓

Compiler

↓

Assembler

↓

Linker

↓

Executable Program

2. Preprocessing

The **preprocessor** processes the preprocessor directives present in the source program before actual compilation.

Main functions:

- Header file inclusion using #include
- Macro expansion using #define
- Conditional compilation using #if, #ifdef, #ifndef, etc.
- Removal of comments

Example

#include <stdio.h>

#define PI 3.14

After preprocessing, the required header contents are included and macros are expanded.

Source Code → Preprocessed Source Code

3. Compilation

The **compiler** translates the preprocessed C program into **assembly language**.

The compiler performs:

- Lexical analysis
- Syntax analysis
- Semantic analysis
- Type checking
- Optimization
- Generation of assembly code

Preprocessed Source Code

↓

Compiler

↓

Assembly Code

The compiler also detects errors such as syntax and semantic errors.

4. Assembly

The **assembler** converts the assembly language instructions into **machine-level object code**.

Assembly Code

↓

Assembler

↓

Object Code

The object code is stored in an **object file**.

The object file contains machine instructions but may contain unresolved references to external functions and libraries.

5. Linking

The **linker** combines:

- Object files
- Required library files
- Other required program modules

It resolves references to external functions and symbols.

For example, when a program uses:

printf();

the linker resolves the reference to the required library implementation.

Object Code + Library Code

↓

Linker

↓

Executable Program

6. Loading

The **loader** is a component of the operating system that loads the executable program from secondary storage into **main memory (RAM)**.

During loading:

- The executable program is placed in memory.
- Required memory areas are prepared.
- Required dynamic libraries are loaded when applicable.
- Addresses and references are prepared for execution.

Executable Program

↓

Loader

↓

Main Memory

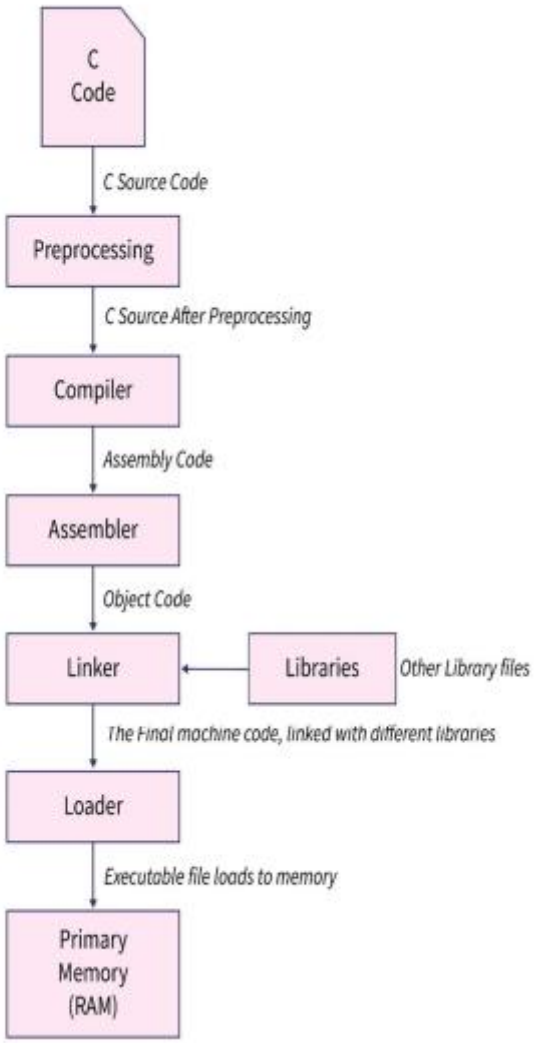
7. Execution

After the program is loaded into memory, the **CPU executes the machine instructions**.

The CPU performs the instruction cycle:

Fetch → Decode → Execute

The instructions are executed sequentially according to the program flow.



Compilation vs Execution

Compilation	Execution
Converts source code into executable/object code	Runs the executable program
Performed by compiler and related tools	Performed by CPU with operating system support
Detects compilation errors	May produce runtime errors
Occurs before execution	Occurs after loading
Produces machine-level/executable code	Produces the actual program result

Data Types, Variables, Constants and Operators in C

1. Data Types

A **data type** specifies the type of data that a variable can store.

Basic Data Types in C

Data Type	Meaning	Example
int	Integer numbers	10, -5, 100
float	Decimal numbers	10.5, 3.14
double	Large/precise decimal numbers	12.345678
char	Single character	'A', 'b', '5'

Examples

int age = 18;

float marks = 85.5;

double price = 1250.5678;

char grade = 'A';

2. Variables

A **variable** is a named memory location used to store a value that can be changed during program execution.

Syntax

data_type variable_name;

Example

int age;

float marks;

char grade;

Variable Initialization

Assigning a value to a variable at the time of declaration.

int age = 18;

float marks = 85.5;

char grade = 'A';

Changing the Value

int age = 18;

age = 20;

The value of age changes from 18 to 20.

Rules for Naming Variables

1. Variable name can contain **letters, digits and underscore (_)**.
2. It cannot start with a digit.
3. Spaces are not allowed.
4. C keywords cannot be used as variable names.
5. C is **case-sensitive**.

Valid

age

student_age

marks1

total_marks

Invalid

1age

student age

float

3. Constants

A **constant** is a value that does not change during program execution.

Types of Constants

Integer Constant

10

25

-50

Floating-Point Constant

3.14

10.5

-2.5

Character Constant

'A'

'b'

'5'

String Constant

"Hello"

"Computer Science"

Using const

The const keyword is used to create a variable whose value cannot be changed.

const int MAX = 100;

const float PI = 3.14;

Example:

const int x = 10;

x = 20; // Error

4. Operators

An **operator** is a symbol used to perform an operation on one or more values.

Example:

a + b

Here, + is an operator.

5. Arithmetic Operators

Arithmetic operators are used to perform mathematical calculations.

Operator	Operation	Example
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b
%	Remainder (Modulus)	a % b

Example

```
int a = 10, b = 3;

printf("%d", a + b);  // 13

printf("%d", a - b);  // 7

printf("%d", a * b);  // 30

printf("%d", a / b);  // 3

printf("%d", a % b);  // 1
```

Important Point

For integer division:

10 / 3 = 3

The decimal part is discarded.

6. Relational Operators

Relational operators are used to **compare two values**.

The result is either **true (1)** or **false (0)**.

Operator	Meaning	Example
==	Equal to	a == b
!=	Not equal to	a != b
>	Greater than	a > b
<	Less than	a < b
>=	Greater than or equal to	a >= b
<=	Less than or equal to	a <= b

Example

```
int a = 10, b = 20;
```

printf("%d", a < b);

Output: 1

Because 10 < 20 is true.

Important Difference

a = 10; // Assignment

a == 10; // Comparison

= is an **assignment operator**, while == is a **relational operator**.

7. Logical Operators

Logical operators are used to combine or reverse conditions.

Operator	**Name**	**Meaning**
&&	Logical AND	Both conditions must be true
!	Logical NOT	Reverses the result

Logical AND &&

(a > 5 && b > 5)

True only when **both conditions are true**.

Logical OR ||

(a > 5 || b > 5)

True when **at least one condition is true**.

Logical NOT !

!(a > 5)

Reverses the result.

Truth Table

A	B	A && B	A || B
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

NOT Truth Table

A	!A
0	1
1	0

Input/Output in C — scanf() and printf()

1. scanf() Function

scanf() is a **formatted input function** used to read data from the **standard input device (keyboard)**.

It is defined in the header file:

#include <stdio.h>

Syntax

scanf("format_string", argument_list);

Example

int a;

scanf("%d", &a);

Here, %d is the format specifier for an integer and &a represents the address of variable a.

Reading Multiple Variables

int a, b, c;

scanf("%d %d %d", &a, &b, &c);

2. printf() Function

printf() is a **formatted output function** used to display data on the **standard output device (console)**.

It is defined in:

#include <stdio.h>

Syntax

printf("format_string", argument_list);

Example

int a = 10;

printf("%d", a);

3. Format Specifiers

Format specifiers specify the **type and format of data** to be read or displayed.

Format Specifier	**Data Type / Use**
%d	Integer
%i	Integer
%u	Unsigned integer
%f	Floating-point number
%lf	Double (scanf)
%c	Single character
%s	String

Format Specifier	Data Type / Use
%o	Octal integer
%x	Hexadecimal integer
%X	Hexadecimal integer (uppercase)
%e	Scientific notation
%g	Fixed or scientific notation

4. scanf() with Different Data Types

Integer

int a;

scanf("%d", &a);

Float

float a;

scanf("%f", &a);

Double

double a;

scanf("%lf", &a);

Character

char ch;

scanf("%c", &ch);

String

char name[20];

scanf("%s", name);

5. printf() with Different Data Types

```
int a = 10;

float b = 12.5;

double c = 25.6789;

char ch = 'A';

char name[] = "Ankit";

printf("%d", a);

printf("%f", b);

printf("%lf", c);

printf("%c", ch);

printf("%s", name);
```

6. Format String

A **format string** contains ordinary characters and format specifiers that determine how input or output is handled.

Example:

```
printf("Age = %d", age);
```

Here:

- "Age = %d" → Format string
- %d → Format specifier
- age → Argument

7. Escape Sequences with printf()

Escape sequences are used to represent special characters.

Escape Sequence	Meaning
\n	New line

Escape Sequence	Meaning
\t	Horizontal tab
\b	Backspace
\r	Carriage return
\\	Backslash
\'	Single quotation mark
\"	Double quotation mark

Example

printf("Hello\nWorld");

Output:

Hello

World

8. Width and Precision in printf()

Field Width

printf("%5d", 25);

The output occupies a field of width 5.

Precision

printf("%.2f", 12.3456);

Output:

12.35

Combined

printf("%8.2f", 12.3456);

Here:

- 8 → Field width
- .2 → Precision
- f → Floating-point value

9. Difference Between scanf() and printf()

scanf()	**printf()**
Formatted input function	Formatted output function
Takes input from keyboard	Displays output on console
Stores input in variables	Displays values of variables
Generally uses & for variables	Does not use & for normal variables
Example: scanf("%d", &a);	Example: printf("%d", a);

Type Conversion in C

Type conversion is the process of converting a value from **one data type to another data type**.

For example, converting an int value into a float value.

Type Hierarchy in C

For basic arithmetic type conversion, you can remember the following order from **lower → higher**:

Level	**Data Type**	**Typical Size**
1	char	1 byte
2	short int	2 bytes
3	int	4 bytes
4	long int	4/8 bytes

Level	Data Type	Typical Size
5	long long int	8 bytes
6	float	4 bytes
7	double	8 bytes
8	long double	8/12/16 bytes

Order to Remember

Char => short => int => long => long long => float => double => long double

Example

int a = 10;

float b = 2.5;

float c = a + b;

Here, int is converted to float before the addition.

Important: This is a simplified teaching hierarchy for common C expressions. **“Higher” does not simply mean “larger size”;** C's actual conversion rules depend on the types involved.

There are **two types of type conversion in C**:

1. Implicit Type Conversion

Implicit conversion is performed **automatically by the compiler**.

Example

int a = 10;

float b;

b = a;

Here, the int value 10 is automatically converted into float.

a = 10

b = 10.0

Example

int a = 10;

float b = 2.5;

float result = a + b;

Here, a is automatically converted from int to float.

General Rule

Smaller/Lower type → Larger/Higher type

Example:

int → float → double

2. Explicit Type Conversion (Type Casting)

Explicit conversion is performed **manually by the programmer** using a **cast operator**.

Syntax

(data_type) expression

Example

int a = 10;

int b = 3;

float result;

result = (float)a / b;

printf("%f", result);

Output:

3.333333

Here:

(float)a convert's a from int to float.

Without Type Casting

```
int a = 5, b = 2;

float result = a / b;

printf("%f", result);
```

Output:

2.000000

Because both a and b are integers, integer division is performed.

With Type Casting

```
int a = 5, b = 2;

float result = (float)a / b;

printf("%f", result);
```

Output:

2.500000

Implicit vs Explicit Conversion

Implicit Conversion	**Explicit Conversion**
Done automatically	Done by programmer
No cast operator required	Cast operator is required
Performed by compiler	Specified by programmer
Example: float b = a;	Example: float b = (float)a;

Implicit: Compiler converts the type automatically.

Explicit: Programmer tells the compiler which type to convert using:

(data_type)

Simple Expressions in C

An expression in C is a combination of constants, variables and operators that produces a value.

Example

a + b

x * 5 + 10

(a + b) / 2

1. Operator Precedence

Operator Precedence determines which operator is performed first when an expression contains multiple operators.

Example

int result = 10 + 5 * 2;

Here, * has higher precedence than +.

So:

10 + 5 * 2

= 10 + 10

= 20

Common Precedence Order

Priority	Operators	Description
1	()	Parentheses
2	* / %	Multiplication, Division, Modulus
3	+ -	Addition, Subtraction
4	< <= > >=	Relational
5	== !=	Equality

Priority	Operators	Description
6	&&	Logical AND
7	?:	Conditional
8	=	Assignment

Higher priority number in the table means lower precedence.

Example

int x = 10 + 20 * 3;

First:

20 * 3 = 60

Then:

10 + 60 = 70

Therefore:

x = 70

2. Associativity

Associativity determines the direction in which operators of the same precedence are evaluated.

There are two directions:

- Left to Right
- Right to Left

Example: Left-to-Right

int x = 20 / 5 * 2;

/ and * have the same precedence.

Their associativity is left to right.

So:

20 / 5 * 2

= 4 * 2

= 8

Not:

20 / 10 = 2

Example: Right-to-Left

Assignment operators have right-to-left associativity.

a = b = c = 10;

Evaluation:

c = 10

b = c

a = b

Therefore:

a = 10

b = 10

c = 10

Precedence vs Associativity

Precedence	Associativity
Determines which operator is performed first	Determines direction of evaluation
Used when operators have different precedence	Important when operators have the same precedence
Example: * before +	Example: * and / are left-to-right

Precedence → Which operator first?

Associativity → Which direction?

Example

int x = 10 + 20 * 3 - 5;

Step 1: * first

10 + 60 - 5

Step 2: + and - have same precedence and are left-to-right

70 - 5

= 65

Answer: 65

When multiplication (*) and division (/) have the same precedence, C follows left-to-right associativity.

So, whichever appears first from the left is executed first.

Example 1

20 / 5 * 2

Both / and * have the same precedence.

From left to right:

20 / 5 * 2

= 4 * 2

= 8

Answer = 8

Example 2

20 * 5 / 2

From left to right:

20 * 5 / 2

= 100 / 2

= 50

Answer = 50

Operator Precedence and Associativity

Operator Precedence

Operator precedence determines **which operator is evaluated first** when an expression contains different operators.

For example:

5 + 2 * 3

* has higher precedence than +, so:

5 + (2 * 3)

= 5 + 6

= 11

Operator Associativity

Associativity determines the **order of evaluation when operators have the same precedence**.

For example:

20 / 5 * 2

/ and * have the same precedence and **left-to-right associativity**:

(20 / 5) * 2

= 4 * 2

= 8

Evaluate the Expression (Imp)

Given:

x = 5 * 2 - 8 / 4 + 3 > 6 && 1

We apply the operators according to their precedence.

Step 1: Multiplication and Division

* and / have higher precedence than + and -.

5 * 2 = 10

8 / 4 = 2

Therefore:

x = 10 - 2 + 3 > 6 && 1

Step 2: Addition and Subtraction

+ and - have the same precedence and are evaluated **left to right**.

10 - 2 + 3

= 8 + 3

= 11

Therefore:

x = 11 > 6 && 1

Step 3: Relational Operator >

11 > 6

This is **true**, so in C:

11 > 6 = 1

Therefore:

x = 1 && 1

Step 4: Logical AND &&

1 && 1 = 1

Therefore:

Final Answer

x = 1

Complete Order of Evaluation

5 * 2 - 8 / 4 + 3 > 6 && 1

↓

10 - 2 + 3 > 6 && 1

↓

8 + 3 > 6 && 1

↓

11 > 6 && 1

↓

1 && 1

↓

1

Final value: x = 1